

0.1 Kiểm thử smart contract

0.1.1 Công cụ và phương pháp kiểm thử

Kiểm thử smart contract được thực hiện bằng phương pháp unit test tự động, sử dụng bộ công cụ sau:

- **Hardhat:** Môi trường phát triển và kiểm thử smart contract, cung cấp mạng blockchain cục bộ (Hardhat Network) cho phép thực thi test mà không tốn gas thực tế. Hỗ trợ loadFixture (tái sử dụng snapshot trạng thái) và time.increase (mô phỏng thời gian trôi qua) [1].
- **ethers.js v6:** Thư viện tương tác với smart contract trong quá trình kiểm thử [2].
- **Mocha + Chai:** Framework tổ chức test case (describe/it) và thư viện assertion. Plugin @nomicfoundation/hardhat-chai-matchers mở rộng Chai với các matcher chuyên dụng cho smart contract như revertedWith, revertedWithCustomError, và emit.
- **solidity-coverage:** Plugin đo lường code coverage cho Solidity, phân tích mức độ bao phủ trên bốn chiều: Statements, Branches, Functions, và Lines.
- **hardhat-gas-reporter:** Plugin báo cáo lượng gas tiêu thụ của từng hàm.
- **TypeChain:** Tự động sinh type definitions từ ABI, đảm bảo type-safety khi viết test bằng TypeScript.

Mỗi test suite sử dụng loadFixture để khởi tạo trạng thái giao thức hoàn chỉnh (deploy toàn bộ smart contract, thiết lập market, cấu hình giá), đảm bảo mỗi test case thực thi độc lập trên một snapshot riêng biệt. Fixture chính deployProtocolWithMarketsFixture tái hiện đầy đủ quy trình deploy 3 phase (Deploy → Wire → Migrate) đã trình bày tại Chương ??, bao gồm deploy toàn bộ 7 smart contract giao thức cùng 3 token MockERC20(WETH, WBTC, USDC) phục vụ kiểm thử, thiết lập ba thị trường với giá và Interest Rate Model tương ứng.

0.1.2 Phạm vi và cấu trúc kiểm thử

Tổng cộng 4 file test kiểm thử 4 nhóm smart contract chính, với tổng số 127 test case. Toàn bộ 127 test case đều pass thành công. Cấu trúc kiểm thử được tổ chức theo bảng 0.1 sau:

File test	Số test	Phạm vi kiểm thử
LendingPool.test.ts	38	initialize (3 test), deposit (5 test), withdraw (5 test), borrow (5 test), repay (3 test), accrueInterest (6 test), preview functions (6 test), initialize revert cases (5 test)
Liquidation.test.ts	31	constructor validation (10 test), isAccountLiquidatable (5 test), calculateSeizeAmount (5 test), liquidate execution (7 test), revert cases (4 test)
InterestRateModel.test.ts	8	constructor validation (3 test), getBorrowRate tại các mức utilization (2 test), getDepositRate (3 test)
ProtocolController.test.ts	50	constructor (6 test), migrateController và update addresses (5 test), price feed (3 test), liquidation params (2 test), my oracle (2 test), pause/unpause (3 test), supportMarket (2 test), unsupportMarket (1 test), setCollateralParams (3 test), setInterestRateModelBatch (3 test), donate (4 test), withdrawTreasury (5 test), rescueToken (4 test), UUPS upgrade (6 test), non-admin revert (1 test)

Bảng 0.1: Kiểm thử tự động smart contract

0.1.3 Chi tiết các nhóm kiểm thử

a, Kiểm thử LendingPool (38 test case)

Đây là test suite lớn nhất, bao phủ toàn bộ chức năng cốt lõi của giao thức:

Khởi tạo (initialize — 3 test): Kiểm tra rằng proxy được khởi tạo đúng tham số (controller, liquidation, priceRouter, collateralFactor); revert khi gọi initialize lần thứ hai (InvalidInitialization); revert khi gọi trực tiếp trên implementation contract (bảo vệ UUPS).

Initialize revert cases (5 test): Kiểm tra deploy proxy revert khi truyền controller address là zero, liquidation address là zero, price router address là zero, collateral factor bằng 0, hoặc collateral factor vượt quá 1.

Deposit (5 test): Kiểm tra cập nhật đúng userBalances.deposited, markets.totalDeposits, depositIndexSnapshot; tích lũy qua nhiều lần deposit; phát đúng event Deposit; hỗ trợ token với decimal khác nhau (USDC 6 decimals, WETH 18 decimals); revert khi market chưa được hỗ trợ, amount bằng 0, hoặc pool đang bị pause.

Withdraw (5 test): Kiểm tra giảm đúng số dư và totalDeposits; trả token về ví người dùng; cap withdrawal tại deposited amount nếu yêu cầu rút vượt quá; cap withdrawal theo collateral constraint khi có khoản vay (đảm bảo health factor

không bị vi phạm); revert tương tự deposit.

Borrow (5 test): Kiểm tra vay trong giới hạn collateral, cập nhật totalBorrows và borrowIndex; hỗ trợ multi-asset collateral; revert khi vượt quá thanh khoản (Not enough liquidity in the market); revert khi vượt quá collateral (Insufficient collateral); revert khi market không hỗ trợ, amount bằng 0, hoặc pool pause.

Repay (3 test): Kiểm tra trả nợ cập nhật đúng state và phát event Repay; cap repayment nếu trả vượt quá số nợ hiện tại và hoàn trả token dư; revert khi chưa có khoản vay (No borrow left), market không hỗ trợ, amount bằng 0, hoặc pool pause.

Accrue Interest (6 test): Kiểm tra lãi suất tích lũy đúng sau khi thời gian trôi qua — totalBorrows, borrowIndex, totalDeposits, depositIndex đều tăng; preview functions phản ánh đúng lãi tích lũy mà không thay đổi state; treasury nhận được phần reserve từ spread borrow-deposit; phát đúng event Accrue; borrower trả lãi nhiều hơn depositor nhận (đảm bảo giao thức có lãi); gọi accrueInterest hai lần trong cùng block là no-op.

Preview Functions (7 test): Kiểm tra previewDeposit, previewBorrow trả về đúng giá trị USD và health factor; previewWithdraw trả về đúng khi có khoản vay đang hoạt động; previewWithdraw trả về type(uint256).max khi không có khoản vay; previewRepay trả về đúng cho partial repay; previewRepay trả về newBorrowedUSD = 0 khi amount vượt quá total borrow; revert khi market không hỗ trợ hoặc amount bằng 0 (kiểm tra 8 assertion trong 1 test).

b, Kiểm thử Liquidation (31 test case)

Constructor validation (10 test): Kiểm tra thiết lập đúng các tham số (priceRouter, lendingPool, controller, liquidationThreshold, closeFactor, liquidationIncentive); revert khi bất kỳ địa chỉ nào là zero (priceRouter, lendingPool, controller); revert khi liquidationThreshold bằng 0 hoặc vượt SCALE; revert khi closeFactor bằng 0 hoặc vượt SCALE; revert khi liquidationIncentive bằng 0 hoặc vượt 0.2e18.

Phát hiện vị thế dưới ngưỡng (isAccountLiquidatable — 5 test): Kiểm tra trả true khi giá collateral giảm khiến vị thế dưới liquidationThreshold; trả false khi không có deposit; trả false khi có deposit nhưng không có borrow; trả false khi vị thế vẫn khỏe; trả true khi lãi suất tích lũy theo thời gian khiến vị thế mất an toàn (mô phỏng 5 năm).

Tính toán lượng tịch thu (calculateSeizeAmount — 5 test): Kiểm tra bao gồm đúng liquidationIncentive; xử lý đúng chênh lệch decimal giữa repay asset (6 decimals) và collateral asset (18 decimals) và ngược lại; tuyến tính theo repay amount; revert khi collateral không có price feed.

Thực thi thanh lý (liquidate — 11 test): Kiểm tra giảm đúng nợ và collateral của borrower; chuyển collateral cho liquidator; cap repay tại $\text{closeFactor} \times \text{currentBorrow}$; không cap repay khi nằm trong giới hạn closeFactor ; cap seize tại borrower deposit balance; phát event LiquidationExecuted; revert khi borrower là zero address; revert khi liquidator address param là zero; revert khi account không liquidatable; revert khi repay amount bằng 0; revert khi gọi seizeCollateral trực tiếp (chỉ cho phép qua Liquidation contract).

c, Kiểm thử InterestRateModel (8 test case)

Constructor (3 test): Kiểm tra constructor thiết lập đúng các tham số immutable (baseRate, rateSlope1, rateSlope2, optimalUtilization, reserveFactor); revert khi optimalUtilization vượt 100%; revert khi reserveFactor vượt 100%.

getBorrowRate (2 test): Kiểm tra trả về đúng giá trị tại 5 mức utilization (0%, 40%, 80%, 90%, 100%) theo công thức Two-Slope Kinked Rate Model; kiểm tra borrow rate tăng đơn điệu theo utilization qua 7 mức.

getDepositRate (3 test): Kiểm tra đúng công thức $\text{borrowRate} \times \text{utilization} \times (1 - \text{reserveFactor})$ tại các mức utilization; deposit rate tăng đơn điệu; deposit rate luôn nhỏ hơn borrow rate do reserveFactor.

d, Kiểm thử ProtocolController (50 test case)

Constructor (6 test): Kiểm tra thiết lập đúng owner và các tham số (lendingPool, myOracle, priceRouter, liquidation, DEFAULT_ADMIN_ROLE); revert khi bất kỳ địa chỉ nào là zero (lendingPool, priceRouter, myOracle, liquidation, admin).

Update addresses (5 test): Kiểm tra migrateController cập nhật controller trên tất cả smart contract liên quan; setLendingPool, setPriceRouter, setMyOracle, setLiquidation cập nhật đúng địa chỉ trên các smart contract phụ thuộc; mỗi hàm revert khi truyền zero address.

Price feed (3 test): Kiểm tra setChainlinkFeed thiết lập Chainlink feed; setMyOracleFeed thiết lập MyOracle feed; removeFeed xóa feed.

Liquidation params (2 test): Kiểm tra setLiquidateParams cập nhật đúng liquidationThreshold, closeFactor, liquidationIncentive; revert khi tham số không hợp lệ (5 assertion: threshold=0, threshold>SCALE, closeFactor=0, closeFactor>SCALE, incentive quá cao).

MyOracle (2 test): Kiểm tra setPrice cập nhật đúng giá và phát event PriceUpdated; revert khi giá bằng 0, asset là zero address, hoặc asset chưa có giá.

Lending Pool functions (26 test):

- Pause/unpause (3 test): pause thành công, non-admin bị revert, unpause thành công.
- supportMarket (2 test): hỗ trợ market mới phát event MarketSupported; cập nhật IRM cho market đã tồn tại.
- unsupportMarket (1 test): bỏ hỗ trợ market phát event MarketUnsupported.
- setCollateralParams (3 test): set thành công phát event; revert khi bằng 0; revert khi vượt SCALE.
- setInterestRateModelBatch (3 test): set IRM cho nhiều asset; revert với IRM zero address; revert với mảng rỗng.
- donate (4 test): donate thành công phát event Donated; revert khi amount = 0; revert khi market chưa hỗ trợ; revert khi pool pause.
- withdrawTreasury (5 test): rút treasury thành công; revert khi amount = 0, recipient = zero, amount vượt balance, hoặc non-admin gọi.
- rescueToken (4 test): rescue thành công; revert khi amount = 0, recipient = zero, hoặc amount vượt surplus.
- Non-admin access control (1 test): non-admin bị revert khi gọi rescueToken.

UUPS Proxy Upgrade (6 test): Kiểm tra upgradePriceRouter — revert khi zero address, revert khi non-admin, upgrade thành công phát event và thay đổi implementation address; upgradeLendingPool — tương tự 3 test.

0.1.4 Kết quả Code Coverage

Code coverage được đo bằng solidity-coverage, kết quả chi tiết chạy lệnh test thể hiện trong hình 0.1, tổng hợp trong bảng 0.2.

Contract	Statements	Branches	Functions	Lines
InterestRateModel	100%	100%	100%	100%
LendingPool	93.55%	76.61%	91.3%	91.22%
Liquidation	100%	78.85%	100%	100%
MyOracle	100%	68.75%	100%	100%
PriceRouter	60%	41.18%	100%	66.67%
ProtocolController	96.15%	75%	95.65%	96.67%
Tổng	93.14%	73.63%	95%	91.93%

Bảng 0.2: Coverage kiểm thử smart contract

127 passing (12s)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	93.08	73.75	94.85	91.86	
InterestRateModel.sol	100	100	100	100	
LendingPool.sol	93.55	76.61	91.3	91.22	... 776,778,820
LendingPoolStorage.sol	100	100	100	100	
Liquidation.sol	100	78.85	100	100	
MyOracle.sol	100	68.75	100	100	
PriceRouter.sol	60	41.18	100	66.67	... 90,91,93,95
PriceRouterStorage.sol	100	100	100	100	
ProtocolController.sol	96.15	75	95.65	96.67	169,170
ProtocolTimelock.sol	100	100	100	100	
contracts/interfaces/	100	100	100	100	
Interfaces.sol	100	100	100	100	
contracts/test/	100	50	100	100	
MockERC20.sol	100	50	100	100	
All files	93.14	73.63	95	91.93	

Hình 0.1: Kết quả chạy lệnh kiểm tra coverage

Nhận xét: Tỷ lệ bao phủ Statements và Lines đạt trên 92%, Functions đạt 95%. Contract PriceRouter có coverage thấp hơn do các nhánh liên quan đến Chainlink Aggregator (yêu cầu mock phức tạp hơn) chưa được kiểm thử đầy đủ — đây là hướng cải thiện trong tương lai. Branch coverage ở mức 74% xuất phát từ hai nhóm nhánh chưa được bao phủ đầy đủ. Nhóm thứ nhất là các nhánh liên quan đến Chainlink Aggregator trong PriceRouter (40% branch coverage): để kiểm thử các nhánh `answeredInRound < roundId` và `updatedAt == 0`, cần mock AggregatorV3Interface trả về các giá trị bất thường — đây là kỹ thuật khả thi nhưng chưa được triển khai trong phạm vi đồ án. Nhóm thứ hai là các nhánh bất biến phòng thủ được kiểm tra tại cả LendingPool lẫn Liquidation (defense-in-depth), vốn chỉ có thể kích hoạt nếu smart contract gọi đến bị compromise — điều không thể tái hiện trong môi trường test thông thường mà không can thiệp trực tiếp vào bytecode.

0.1.5 Kết quả Gas Report

Gas report được sinh bởi hardhat-gas-reporter, cho biết lượng gas tiêu thụ của các hàm chính, thể hiện trong bảng 0.3 và bảng 0.4 thể hiện chi phí deploy của các smart contract:

Hàm	Min (gas)	Max (gas)	Avg (gas)	Số lần gọi
deposit	80,832	168,290	136,393	41
withdraw	57,997	96,766	84,541	7
borrow	84,044	151,584	127,718	19
repay	48,764	71,752	59,217	4
accrueInterest	30,345	85,069	60,789	9
liquidate	129,430	209,447	164,849	7

Bảng 0.3: Gas report kiểm thử smart contract các hàm chính

Contract (Deploy)	Gas	% Block limit
LendingPool	3,396,978	11.3%
ProtocolController	1,474,420	4.9%
ProtocolTimelock	1,356,288	4.5%
Liquidation	1,061,830	3.5%
PriceRouter	914,408	3.0%

Bảng 0.4: Gas report kiểm thử smart contract deploy

Nhận xét: Các hàm nghiệp vụ chính có gas cost hợp lý — deposit trung bình 136k gas, liquidate trung bình 165k gas. Chi phí deploy lớn nhất là LendingPool (11.3% block gas limit) do logic phức tạp, nhưng vẫn nằm trong giới hạn cho phép và chỉ phải deploy một lần.

0.2 Kiểm thử Backend

0.2.1 Công cụ và phương pháp kiểm thử

Kiểm thử backend được thực hiện bằng phương pháp unit test tự động với chiến lược cô lập hoàn toàn (isolation testing) — mọi phụ thuộc hạ tầng (database, Redis, blockchain provider) đều được thay thế bằng mock objects. Bộ công cụ sử dụng:

- **Vitest:** Framework kiểm thử hiện đại cho TypeScript/ESM, hỗ trợ native ES modules, mock functions (vi.fn()), và tích hợp coverage qua V8 provider.
- **Mock Factory Pattern:** Hệ thống tạo mock được tổ chức trong thư mục test/shared/mocks.ts, cung cấp các factory function createMockDbClient(), createMockRedis(), createMockLogger().
- **Domain Fixtures:** File test/test-helpers/domain.ts cung cấp các factory function tạo dữ liệu mẫu (createUserRow, createAssetRow, createSessionRow, createTransactionRow) với giá trị mặc định hợp lý.
- **Error Assertion Helper:** Hàm expectAppErr(promise, errorCode) đóng gói logic kiểm tra rằng một Promise reject với đúng AppErr và mã lỗi mong đợi.

0.2.2 Phạm vi và cấu trúc kiểm thử

Tổng cộng 9 file test kiểm thử 7 module của hệ thống backend, với tổng số 71 test case (toàn bộ pass), được trình bày bằng bảng 0.5 và hình 0.2.

File test	Số test	Phạm vi kiểm thử
signature.service.test.ts	9	Tạo SIWE message, xác minh chữ ký Ethereum, quản lý nonce Redis, rate-limit nonce requests
session.service.test.ts	8	Tạo session, refresh token, revoke session, xử lý session hết hạn/bị thu hồi
token.service.test.ts	8	Sinh JWT access token, xác minh token (hợp lệ, hết hạn, sai secret, malformed, sai tokenType, thiếu claims)
asset.service.test.ts	8	Lấy danh sách/chi tiết tài sản, lấy cấu hình thị trường, xử lý lỗi DB, validate Ethereum address
blc-worker.handler.test.ts	19	Xử lý event Deposit/Withdraw/Borrow/Repay/Accrue/Liquidation/Donate/TreasuryWithdrawn/MarketUnsupported, idempotency (skip duplicate)
transaction.service.test.ts	5	Lấy lịch sử giao dịch, validate input, xử lý user không tồn tại
user.service.test.ts	7	Lấy thông tin user, dashboard detail (user + assets), lấy email, xử lý user không tồn tại, validate address
rate-limit.service.test.ts	4	ensureCooldown (SET NX), ensureAttemptLimit (INCR + TTL), xử lý khi vượt giới hạn
validate.test.ts	3	Checksum Ethereum address, reject địa chỉ không hợp lệ, reject zero address

Bảng 0.5: Kiểm thử tự động backend

```
hoangdung@DESKTOP-ELNTE38:~/lending-pool-prj3/backend$ npm run test

> backend@1.0.0 test
> vitest run

RUN v4.1.7 /home/hoangdung/lending-pool-prj3/backend

✓ test/unit/modules/blockchain-worker/blc-worker.handler.test.ts (19 tests) 147ms
✓ test/unit/modules/users/user.service.test.ts (7 tests) 49ms
✓ test/unit/modules/auth/signature.service.test.ts (9 tests) 217ms
✓ test/unit/modules/auth/session.service.test.ts (8 tests) 28ms
✓ test/unit/modules/auth/token.service.test.ts (8 tests) 20ms
✓ test/unit/modules/transactions/transaction.service.test.ts (5 tests) 21ms
✓ test/unit/shared/rate-limit.service.test.ts (4 tests) 9ms
✓ test/unit/shared/validate.test.ts (3 tests) 6ms
✓ test/unit/modules/assets/asset.service.test.ts (8 tests) 36ms

Test Files 9 passed (9)
Tests 71 passed (71)
Start at 21:43:58
Duration 2.15s (transform 958ms, setup 0ms, import 3.76s, tests 534ms, environment 1ms)
```

Hình 0.2: Kết quả chạy lệnh kiểm thử backend

0.2.3 Chi tiết các nhóm kiểm thử quan trọng

a, Kiểm thử xác thực (Authentication)

Module xác thực được kiểm thử ở cả 3 tầng: SignatureService (tạo message + verify chữ ký), SessionService (quản lý session), TokenService (sinh/xác minh JWT).

SignatureService sử dụng ethers.Wallet.createRandom() để tạo ví ngẫu nhiên trong mỗi test, ký message thực sự bằng wallet.signMessage(), sau đó gọi verifyMessage để kiểm tra luồng xác thực end-to-end. Test kiểm tra: nonce được lưu Redis và xóa sau khi verify thành công; chữ ký sai bị reject; nonce hết hạn bị reject; user được tạo tự động lần đầu đăng nhập qua findOrCreate.

TokenService kiểm tra JWT được sinh đúng format (issuer, audience, subject, expiration, custom claims); token hết hạn, sai secret, malformed, sai tokenType, hoặc thiếu claims đều bị reject với mã lỗi Unauthorized.

SessionService kiểm tra tạo session trả về đầy đủ accessToken và refreshToken; refresh token hợp lệ sinh access token mới; session không tồn tại, bị revoke, hoặc hết hạn đều bị reject với mã lỗi tương ứng (SessionNotFound, SessionRevoked, SessionExpired).

b, Kiểm thử BLC Worker Handler

Đây là test suite quan trọng nhất phía backend, kiểm thử logic xử lý blockchain events. Mỗi loại event được kiểm thử theo 3 kịch bản:

- Happy path: Event được xử lý thành công — tạo transaction record, cập nhật userAsset, cập nhật asset state.
- Idempotency: Event duplicate (cùng transactionHash + logIndex) bị skip mà không throw error — kiểm tra rằng lỗi unique constraint được bắt và xử lý đúng.

Kiểm thử Accrue event còn bao gồm: phát hiện treasury balance drift (cảnh báo khi giá trị thực tế khác giá trị kỳ vọng), và skip duplicate accrue log.

c, Kiểm thử Rate Limiting

Kiểm tra cơ chế rate limiting dựa trên Redis SET NX (cooldown) và INCR (attempt counting): lần gọi đầu tiên thành công (SET NX trả về "OK"); lần gọi thứ hai trong khoảng cooldown bị reject với RateLimitExceeded; attempt count vượt giới hạn bị reject.

0.2.4 Kết quả kiểm thử

Toàn bộ test case được chạy bằng lệnh `npm run test` (sử dụng Vitest). Kết quả: tất cả test case pass thành công. Thời gian thực thi trung bình dưới 5 giây nhờ chiến lược mock hoàn toàn, không phụ thuộc vào hạ tầng thực.

0.3 Kiểm thử Frontend

Phần frontend được kiểm thử theo phương pháp kiểm thử hộp đen (Black-box Testing), tập trung vào hành vi và kết quả hiển thị thay vì cấu trúc mã nguồn. Các tiêu chí kiểm thử:

- Tính đúng đắn của các chức năng giao diện: kết nối ví, deposit, withdraw, borrow, repay, xem lịch sử giao dịch, xem danh sách thanh lý.
- Dữ liệu hiển thị phản ánh đúng trạng thái blockchain: số dư cập nhật real-time qua view function mỗi 15 giây, danh sách liquidatable positions cập nhật qua WebSocket.
- Xử lý lỗi: hiển thị thông báo khi ví chưa kết nối, transaction bị reject, hoặc số dư không đủ.

Các công cụ kiểm thử:

- Trình duyệt web: Google Chrome, Microsoft Edge — kiểm thử trên nhiều trình duyệt để đảm bảo tương thích.
- MetaMask: Kiểm thử tương tác ví (kết nối, ký message SIWE, ký giao dịch, chuyển mạng).
- Etherscan (Sepolia): Xác minh trạng thái giao dịch on-chain, so khớp event logs với dữ liệu hiển thị trên giao diện.

Ngoài ra, kiểm thử tích hợp end-to-end được thực hiện thủ công để xác minh luồng hoàn chỉnh: event từ blockchain → blc-indexer phát hiện → RabbitMQ → blc-worker xử lý → database cập nhật → frontend hiển thị đúng.